PROJECT-1

Problem Title:-16-bit ALU Design with Basic Testbench and Flag Support

Project Description:-Design a 16-bit ALU capable of performing basic arithmetic and logical operations along with generation of essential status flags.

The goal is to evaluate:

Fundamental RTL design skills

Basic verification ability

Code organization and documentation

Problem Statement:-

The ALU should be capable of performing the following operations (minimum requirements):

Arithmetic Operations

- Addition (A + B)
- Subtraction (A − B)

Logical Operations

- AND
- OR
- XOR
- NOT (unary operation on A)

Shift / Rotate Operations

- Logical Left Shift
- Logical Right Shift

Status Flags (Mandatory)

The ALU must generate the following status flags:

- Z (Zero): Set when the result is zero
- C (Carry): Carry generated in addition or borrow in subtraction
- N (Negative): Most Significant Bit (MSB) of the result (sign bit)
- V (Overflow): Indicates signed overflow condition

1.Intoduction:-

An **Arithmetic Logic Unit (ALU)** is a fundamental component of digital systems and processors. It performs arithmetic and logical operations on binary data.

In this project, a **16-bit ALU** is designed using Verilog HDL that supports multiple operations and generates status flags. The design is verified using a **testbench** and analyzed through simulation waveforms.

The goal of this project is to design a **16-bit Arithmetic Logic Unit (ALU)** that can perform a set of arithmetic, logical, and shift operations on binary inputs.

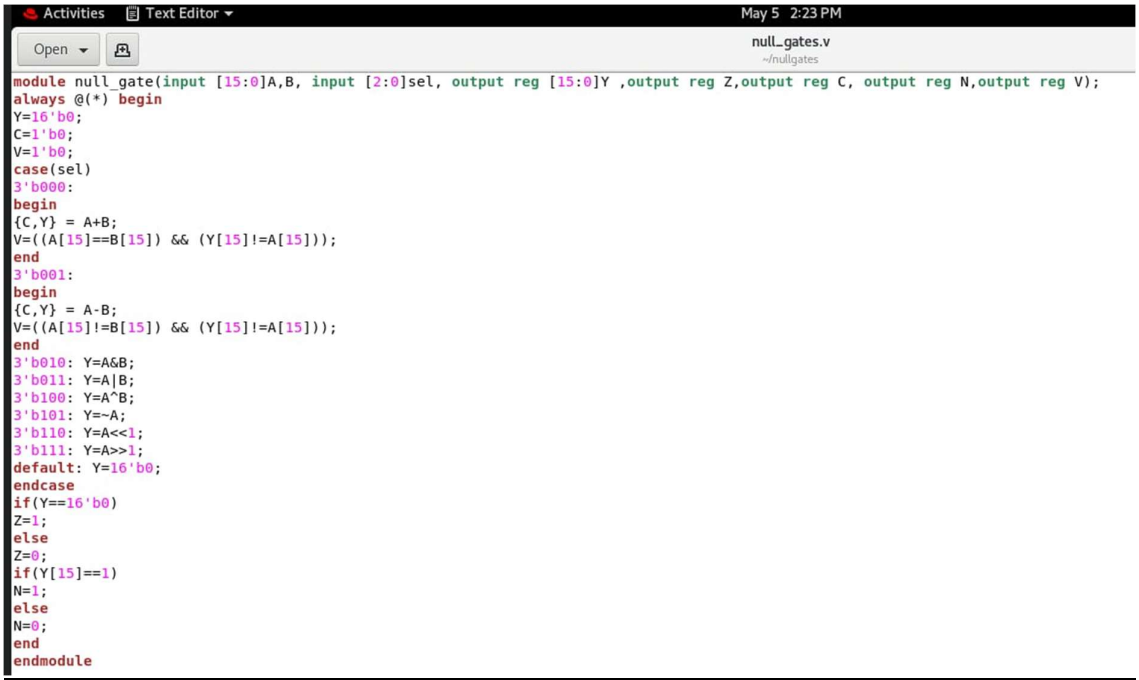
The ALU must also generate **status flags** such as Zero, Carry, Negative, and Overflow, which are essential in processor operations.

In addition to design, the functionality must be verified using a **testbench**, and correctness must be validated using **simulation waveforms**.

2. Objective

- To design a **16-bit ALU** using Verilog HDL
- To implement **arithmetic, logical, and shift operations**
- To generate **status flags (Z, C, N, V)**
- To verify the design using a **testbench**
- To analyze correctness using **waveforms**

3.Code :-




```
module null_gate(input [15:0]A,B, input [2:0]sel, output reg [15:0]Y ,output reg Z,output reg C, output reg N,output reg V);
always @(*) begin
Y=16'b0;
C=1'b0;
V=1'b0;
case(sel)
3'b000:
begin
{C,Y} = A+B;
V=((A[15]==B[15]) && (Y[15]!=A[15]));
end
3'b001:
begin
{C,Y} = A-B;
V=((A[15]!=B[15]) && (Y[15]!=A[15]));
end
3'b010: Y=A&B;
3'b011: Y=A|B;
3'b100: Y=A^B;
3'b101: Y=~A;
3'b110: Y=A<<1;
3'b111: Y=A>>1;
default: Y=16'b0;
endcase
if(Y==16'b0)
Z=1;
else
Z=0;
if(Y[15]==1)
N=1;
else
N=0;
end
endmodule
```

4.Testbench:-

```
module null_gate_tb;
reg [15:0]A,B;
reg [2:0]sel;
wire [15:0]Y;
wire Z;
wire C;
wire N;
wire V;
null_gate uut(.A(A),.B(B),.sel(sel),.Y(Y),.Z(Z),.C(C),.N(N),.V(V));
initial begin
A=16'd10;
B=15'd5;
sel=3'b000;
#10;
$display("ADD:Y=%d Z=%b C=%b N=%b V=%b",Y,Z,C,N,V);
sel=3'b001;
#10;
$display("SUB:Y=%d Z=%b C=%b N=%b V=%b",Y,Z,C,N,V);
sel=3'b010;
#10;
$display("AND:Y=%d",Y);
sel=3'b011;
#10;
$display("OR:Y=%d",Y);
sel=3'b100;
#10;
$display("XOR:Y=%d",Y);
sel=3'b101;
#10;
$display("NOT:Y=%d",Y);
sel=3'b110;
#10;
$display("LSHIFT:Y=%d",Y);
sel=3'b111;
#10;
$display("RSHIFT:Y=%d",Y);
$finish;
end
endmodule
```

---

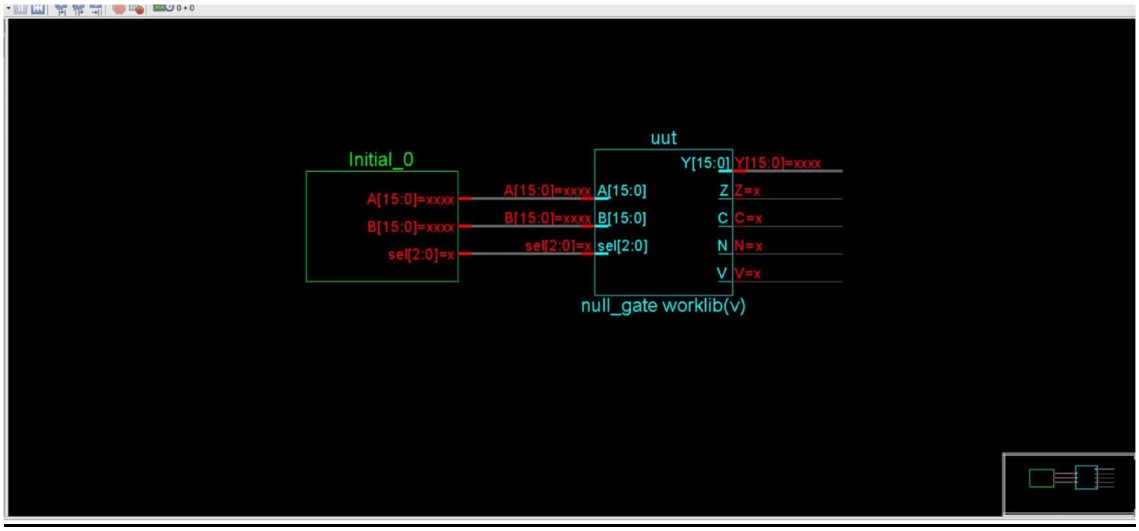
5. Features of the ALU

- 16-bit data processing
- Supports multiple operations using opcode
- Generates 4 status flags:
    - Zero (Z)
    - Carry (C)
    - Negative (N)

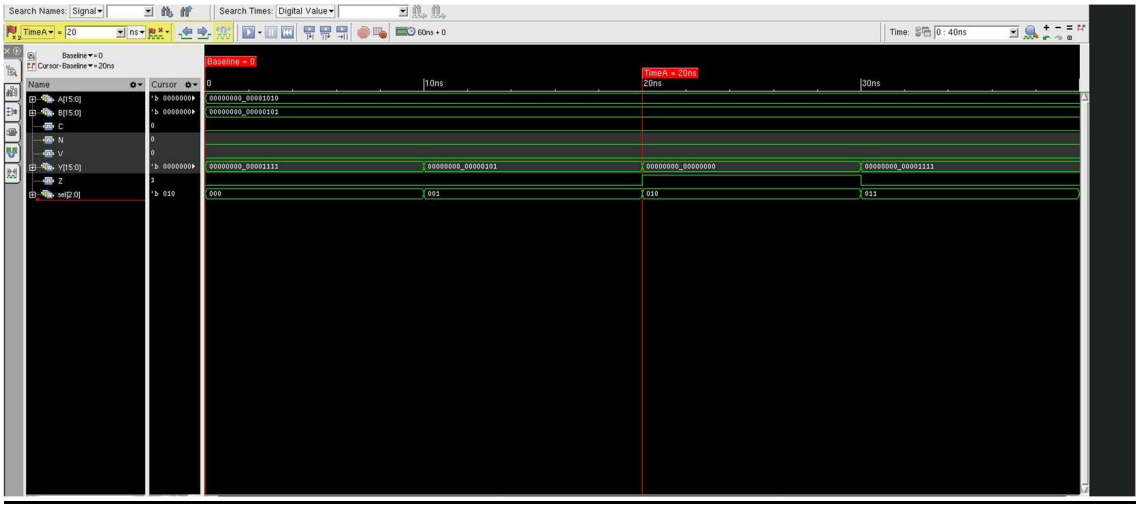
- Overflow (V)

- Combinational design (no clock required)
- Easy to extend (parameterizable design possible)

6.Block Diagram:-



7.Waveform:-



8. Approach / Logic Used

The ALU is implemented using a **combinational design approach** in Verilog.

- A **case statement** is used to select operations based on the opcode
- Each operation (ADD, SUB, AND, etc.) is mapped to a unique opcode
- Arithmetic operations use built-in Verilog operators

- Flags are generated based on the result:
    - **Zero (Z)** → result equals zero
    - **Negative (N)** → MSB of result
    - **Carry (C)** → from addition/subtraction
    - **Overflow (V)** → detected using sign logic

The design ensures all outputs are updated instantly for any input change using always @(*).

8.Repository Link:-

https://github.com/nishitap2006-gif/rtl